

이민혁 클라이언트 개발자

기술 소개서

Contact.

☎ 010-2058-3532 ✉ seif4688@gmail.com

목차

0. 프로필

1. WinAPI Don't Starve 모작

[C++, WinAPI] - 최적화 이슈 해결, 에디터 툴

2. File Tower Defence

[C#, Unity] - 좌표계 변환 이슈 해결, 유닛 이벤트 처리

3. Car Racing Simulation

[C#, Unity, ML-Agent] - 학습환경 및 보상 로직 설계, 역주행 이슈 해결

4. World First Kill

[C#, Unity] - CSV데이터 다운로드/파싱, Seed/Token세이브 로드

5. 기타 언급 및 프로젝트

프로필

개인 정보

이름

이민혁

이메일

seif4688@gmail.com

번호

010-2058-3532

학력/교육 이력

2021.03 ~ 현재

동명대학교 재학

활동

2024 ~ 현재

UpTrand 동아리 활동

2025.03 ~ 2026.03

Uptrand 동아리 회장 운영

2026.01

MACH GAME JAM

2026.01

GGJ 글로벌 게임잼

2025

서울 코믹 월드 인디 부스 참여

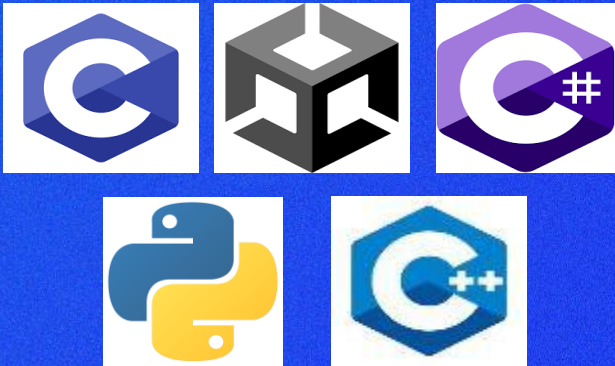
2025

지스타 학교 부스 전시 참여,
인디쇼케이스 부스 참여

2024

지스타 학교 부스 전시 참여

기술



Certificate

2023.04

정보처리기능사

2024.09

프롬프트 디자이너

Award

2025

제3회 **TU Game Challenge** 우수상

2025

지스타 인디어워즈 **Best Experimental Game**

2025

창업아이디어 경진대회 우수상

2025

부산지역인재 장학금 IT분야 수여

2025

대동장학재단 장학금 수여

2021

TU-SEED 리빙랩 경진대회 입상

WinAPI 돈스타브 모작

플랫폼	PC(Windows)		
개발 인원	1인(개인)	개발 기간	2025. 12. 05 ~ 2026. 02. 04 (대략 2달)
개발 도구	C++, WinAPI	동영상 링크	https://www.youtube.com/watch?v=rkKOD-KXINg
요약	플레이어는 마우스 조작을 통하여 이동, 상호작용을 할 수 있으며, 최종 목표인 최종 보스 몬스터를 쓰러트리는 것을 목적으로 한다. 최종 보스 몬스터를 쓰러트리기 위해서 플레이어는 재화를 수집하여, 한정된 자원으로 아이템을 만들어 보스에 도전할 수 있다.		
동기	게임 엔진을 의존하지 않고, WinAPI를 이용하여 완전 밑바닥부터 게임을 만들어보고자 모작에 도전하게 되었다. 게임 실행의 전체적인 파이프라인을 파악하고, 이를 스스로 설계해보고자 하여 시작하게 되었다.		

게임 개발 내용

오브젝트가 늘어날 수록 발생하는 오브젝트 순회의 부담감의 문제점

맵이 커지고, 오브젝트 수가 1,000 ~ 10,000개 늘어날 수록 뷰포트안에 들어오는 객체 수를 검사해야 하는 양이 많아져, 반복문을 그 만큼 돌게 되고, 성능이 나빠질 것이 우려되었다.

또한 뷰포트만 아니라, 오브젝트의 콜라이더 검사, 상호작용 처리 로직에서도 수 많은 객체의 루프에 대한 똑같은 딜레마가 생긴다.

```
for (GameObject* obj : m_worldObjects) {  
    if (!obj || !obj->IsEnabled() || obj->IsDead()) continue;  
  
    if (cameraManager->IsObjectInViewport(obj)) {  
        outObjects.push_back(obj);  
    }  
}
```



월드 오브젝트 객체가 많아질
수록 순회의 부담감이 증가

게임 개발 내용

문제에 대한 해결 기획

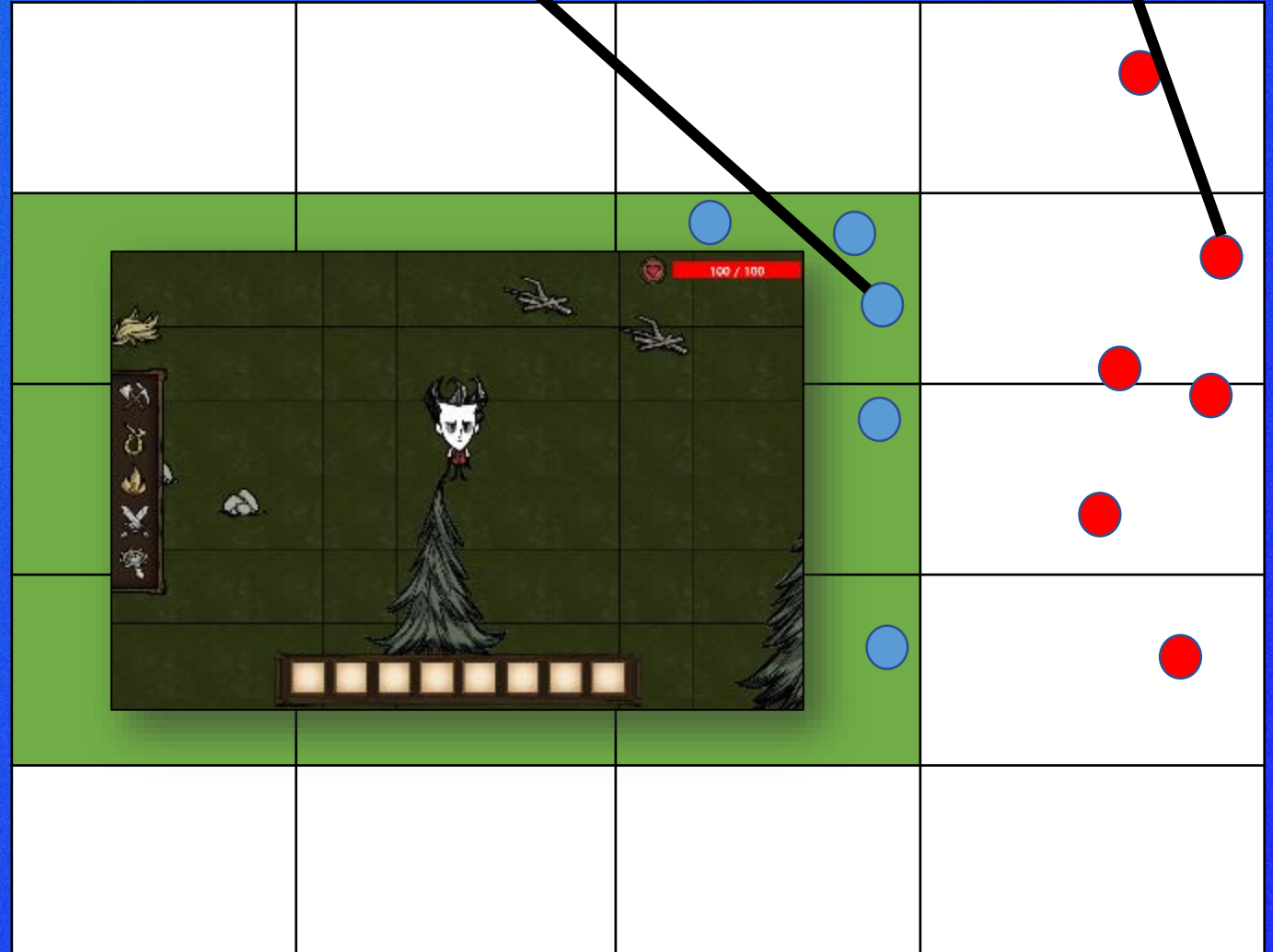
공간을 나눠서, 특정 공간에 오브젝트를 검사하고, 그 오브젝트들만 렌더링하는 방안을 생각할 수 있었음

↓
뷰포트 영역 기준으로 그리드를 업데이트 하고, 그리드안에 위치해 있으며 뷰포트 안에 존재하는 오브젝트를 찾는다.

↓
그렇게 찾은 오브젝트들만 렌더링을 하도록 설계 한다.

그리드가 뷰포트 안에 겹쳐지는(초록색 범위) 오브젝트를 후보군으로 검사함

그리드 범위 밖이므로 후보군 목록에 넣어 검사하지 않음



게임 개발 내용

문제 해결 과정 설계

수정 (핵심코드 설명 필요)

GameObject

- + Transform Component
- + GridX, GridY 좌표
- + Render

Transform 컴포넌트가
SetTransform으로 오브젝트
가 움직인다면, GridX,Y 좌표
를 살펴봐 최신화 한다.

ObjectManager

- + UpdateObjectGrid
- + QueryObjectsInRectArea
- + AddGrid, RemoveGrid

오브젝트마다 그리드의 업데이트를 해주고, 맵상에 그리드로 배치, 삭제를 전담함.

특정 공간에 오브젝트가 존재하는지 탐색함

CameraManager

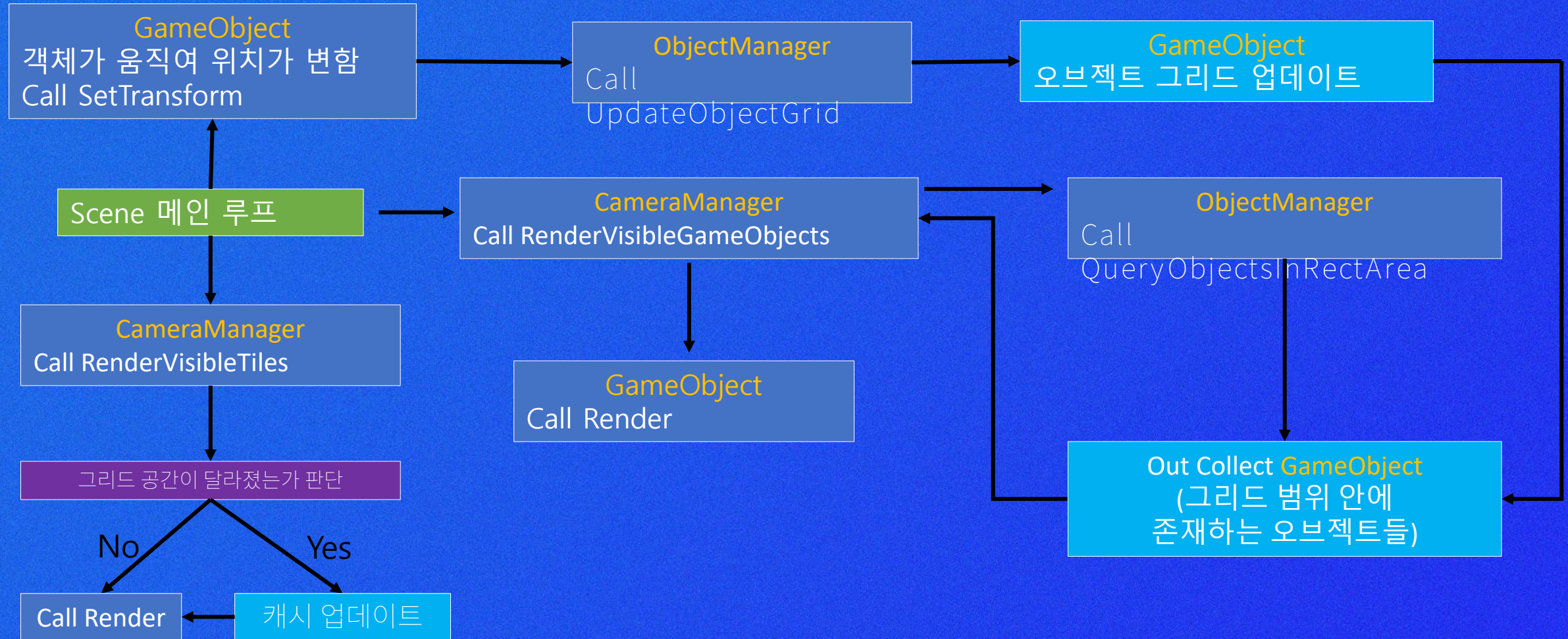
- + RenderVisibleGameObjects
- + RenderVisibleTiles

현재 뷰포트 범위에 보이는
오브젝트와, 타일을 렌더링
함수를 호출함

게임 개발 내용

문제 해결 과정 설계

전체적인 파이프라인



게임 개발 내용

문제 해결 결과

CullVisibleGameObjects	: 0.125ms
RenderVisibleGameObjects	: 0.014ms
RenderVisibleTiles	: 0.115ms
[통계]	
전체 월드 오브젝트 수	: 1081
렌더링 엔티티 수	: 8

모든 오브젝트를 대상으로 Culling검사

CullVisibleGameObjects	: 0.014ms
RenderVisibleGameObjects	: 0.012ms
RenderVisibleTiles	: 0.117ms
[통계]	
전체 월드 오브젝트 수	: 1081
렌더링 엔티티 수	: 3

그리드 공간안에 들어가 있는 오브젝트를 대상으로 Culling검사

약 10배 성능개선 효과를 보였음
오브젝트가 더 많아진다면,
더 효과적인 성능 개선을 기대해 볼 수 있음

File Tower Defence

플랫폼	PC(Windows)	장르	로그 라이크 타워 디펜스
개발 인원	4인(기획1,플밍2,아트1)	개발 기간	2025. 05 ~ 현재
개발 도구	C#, Unity	동영상 링크	
요약	Windows OS와 파일, 확장자를 컨셉으로 하여, 주어진 경로에 바이러스를 막는 디펜스 게임이다. 플레이어는 라운드를 클리어하여 얻은 재화를 통해 더 상위인 파일을 해금하거나 영구적으로 파일들의 위력을 높일 수 있다.		
역할	파일/바이러스/UI 입력 및 상호작용 이벤트 시스템 설계 버프 파일 유닛의 버프 부여/해제 로직 설계 그리드 기반 오브젝트 배치 시스템 설계		

게임 개발 내용

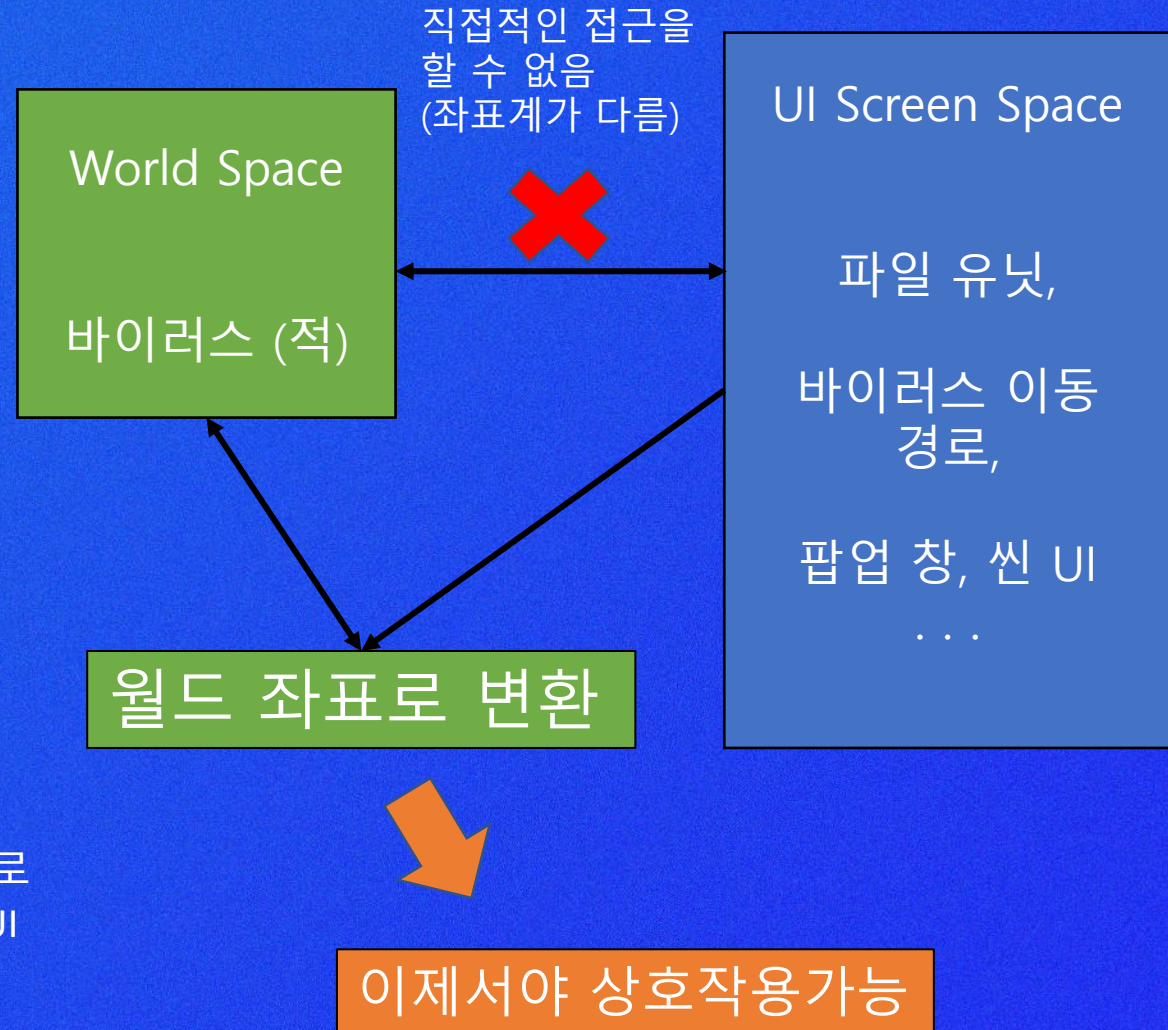
UI 기반 설계로 인한 월드 좌표계와의 변환이 까다로움을 느낌

초기 개발 단계에서, 우리 Windows OS에 존재하는 파일의 드래그&드랍의 배치 느낌을 모티브를 삼았다.

파일들을 UI 오브젝트로 만들어, 이를 "UI Slot을 만들고, 배치하면, 되지 않을까?" 라는 생각을 하고 만듦

바이러스가 UI(RectTransform) 객체의 위치를 비교하거나 찾고자 할 때
빈번한 좌표계 변환으로
로직의 복잡도가 올라갔음

다른 Canvas나, 부모 RectTransform로 이동했을 때, Canvas 안에 수 많은 UI 자식들이 존재했을 때
성능적인 부하도 존재함



게임 개발 내용

문제에 대한 해결 기획

모든 파일을 오브젝트로 바꾸고, 오브젝트를 배치할 수 있도록 하기 위해, 오브젝트 전용 그리드를 만들어 버리자.
유연한 그리드 설정을 위해서 전용 레이아웃을 만들자.

해당 구조로 바꾸고 좋아진 점

1. 좌표계 통일,
2. 그리드 기반 버프 부여/해제,
3. 유연한 그리드 배치 및 생성
4. Canvas ReBuilding에 대한 성능 부하 감소

GridManager

+TrySetUnitAtGrid

그리드에 유닛 배치

+GridIndexToWorld

인덱스로 월드 좌표 접근

+WorldToGridIndex

월드 좌표로 인덱스 접근

GameObjectGridLayout

+Grid columns,rows / cellsize,spacing,padding

그리드 정보 정의

+FitToScreen

화면 크기별 그리드 크기 조정

+AddGridObject

그리드 생성

FileGrid

+fileUnit

파일 유닛 소유

+activeBuffSources

파일 버프 소유

+GridX,Y

그리드 x,y 위치값

+ApplyGridBuffs

그리드에 존재하는 버프 부여

+ApplyBuffActive

그리드에게 버프 부여

게임 개발 내용

문제 해결 과정 설계

Grid Search (x,y)

GridManager
Call WorldToGridIndex

GridManager
Call FindClosestGridInRange

Return FileGrid

특정 FileGrid에
파일 유닛 배치

GridManager
Call TrySetUnitAtGrid

FileGrid
Call SetFileUnit

FileGrid
Call ApplyGridBufs(true)
Call ApplyBuffActive(true)

그리드에서 제거

GridManager
Call TryRemoveUnit

FileGrid
Call RemoveFileUnit

FileGrid
Call ApplyGridBufs(false)
Call ApplyBuffActive(false)

GridManager
초기화

GameObjectGridLayout
Call AddGridObject
Call FitToScreen

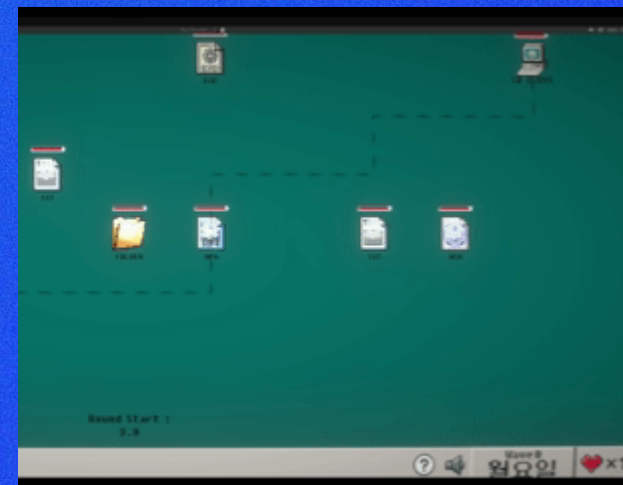
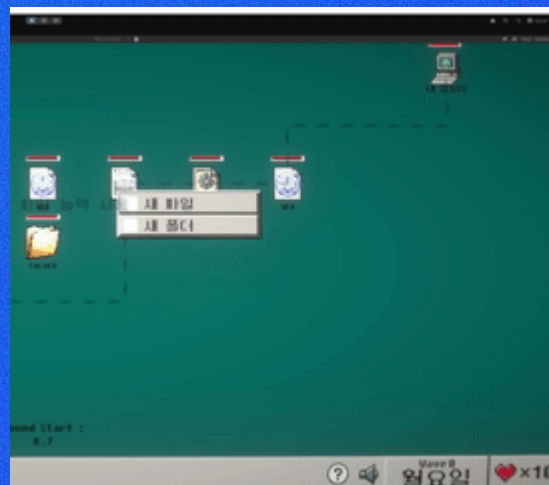
FileGrid Object 생성

게임 개발 내용

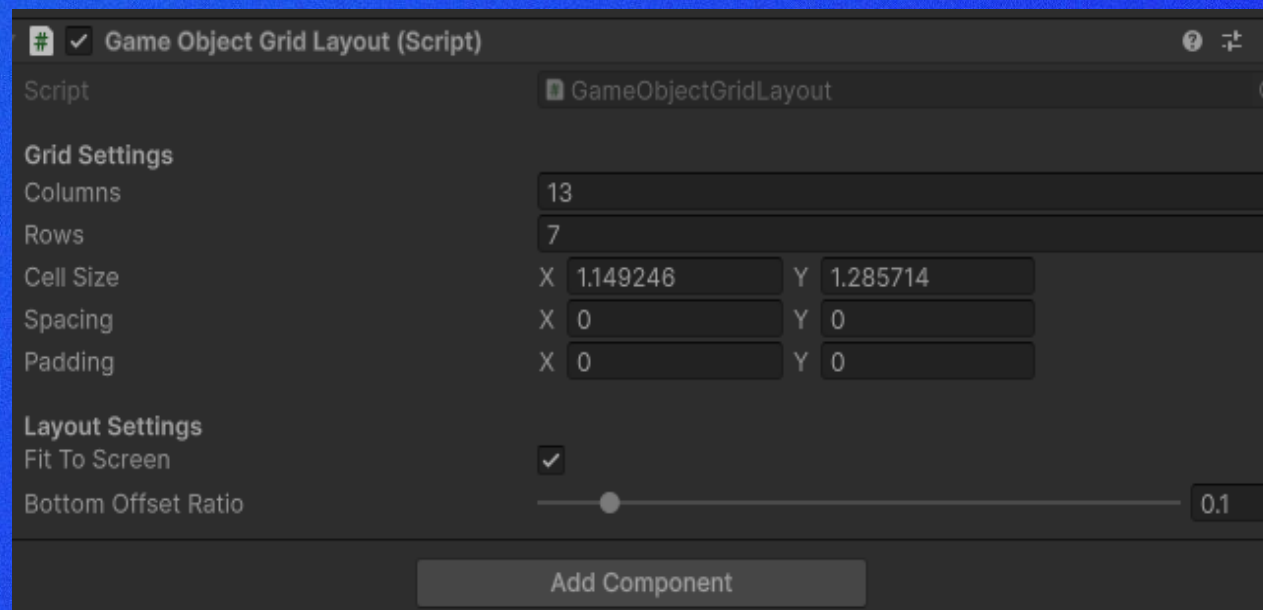
문제 해결 결과

그리드 공간 별로 유닛 배치 및 영역 밖에 있는 파일 유닛은 근접한 그리드로 점유 하도록 설계

이 결과 Windows 바탕화면 아이콘을 드래그&드랍하는 느낌을 유지할 수 있었음



유연한 그리드 설정 수정, 그리드 셀 크기 자동 조정 기능을 가진 레이아웃 클래스



게임 개발 내용

유닛 이벤트 처리

InputManager

InteractionHandler

IInteractable

File Unit

Virus Unit

UI Element

InputManager: 마우스, 키보드 입력을 처리하는 매니저 클래스
Raycast를 통해 현재 UI위에 마우스 포인터가 존재하는지 체크
입력 상태에 따른 분기를 나눠 Hover, Click, Drag 상태를 체크

InteractionHandler: 실질적인 이벤트를 날려주는 핸들러 클래스
InputManager로 부터 받은 Hover, Click, Drag 이벤트를 호출
객체들의 상태에 따라서 논리적 동작을 제어

IInteractable: 인터페이스로, 해당 이벤트 함수를 선언한다.
상호작용 이벤트가 필요한 객체에게 상속하여 이벤트 함수를 각 객체마다
구현할 수 있다.

Car Racing Simulation

플랫폼	PC(Windows)	장르	시뮬레이션
개발 인원	2인(기획1,플밍1)	개발 기간	2025. 09 ~ 현재
개발 도구	C#, Unity, ML-Agent	동영상 링크	
요약	에이전트끼리 레이싱 경주를 하는 것을 관찰하고 플레이어는 그 사이 1등 에이전트를 예측하여 베팅하는 시스템을 가진 시뮬레이션 게임 ML-Agent 툴킷을 이용한 에이전트를 통해 예측 불가능한 상황을 만들어 베팅하는 재미와 예측하는 재미를 챙기기 위해 기획함		
역할	ML-Agent 학습 환경 및 보상 로직 설계 에이전트 네비게이션 시스템 개발		

게임 개발 내용

차량이 역주행 하는 문제점



장애물 충돌



장애물을 부딪치거나, 후진을 했을 때, 일방적인 주행중일 때도, 차가 회전하여 방향이 트랙방향이 아니라, 반대 방향으로 역주행을 하는 경우가 존재했음

이때 다시 트랙 방향으로 나아갈 수 있는 수단이 필요했음

게임 개발 내용

문제에 대한 해결 기획

일종의 네비게이션 역할을 하는 기능을 넣어 줘야겠다.
트랙 방향과 멀어지거나, 특정 구간을 오랫동안 통과하지 못하면 네거티브를 부여해줘야 겠다.



해당 구간으로 이동하여
통과하면 파지티브 부여



구간과 멀어질 수록 네거티브, 해
당 구간을 오랫동안 통과하지 못
하면, 지속적인 네거티브 부여

게임 개발 내용

문제에 대한 해결 기획

1. 트랙 구간(체크포인트)을 만들어야 한다. 이를 위해 유니티의 Spline 기능을 활용해서, 트랙 경로를 그려주고, 그 경로 사이에 구간을 넣어준다.
2. 일일이 구간을 지정해서 넣어주면 매우 번거롭고, 귀찮기 때문에 이를 자동으로 구간을 깔아주는 기능을 만든다.
3. 해당 구간을 넘어갈 때, 그리고 넘어가지 못하는 에이전트에게 보상값을 넘겨주기 위한 로직을 짠다.

게임 개발 내용

문제 해결 과정 설계

SplineToCheckpointGenerator

TrackData의 트랙 정보를 자동으로 채워주는 툴

+GenerateCheckpoints

+GenerateStartpoints

ContextMenu Attribute로 실행

TrackData

트랙에 정보를 저장

+Set Checkpoint, Startpoint, Finishpoint

+Get Checkpoint, Startpoint, Finishpoint

CheckPoint

+배열 index

+OnTriggerEnter (파지티브 값)

게임 개발 내용

문제 해결 과정 설계

CarProgress

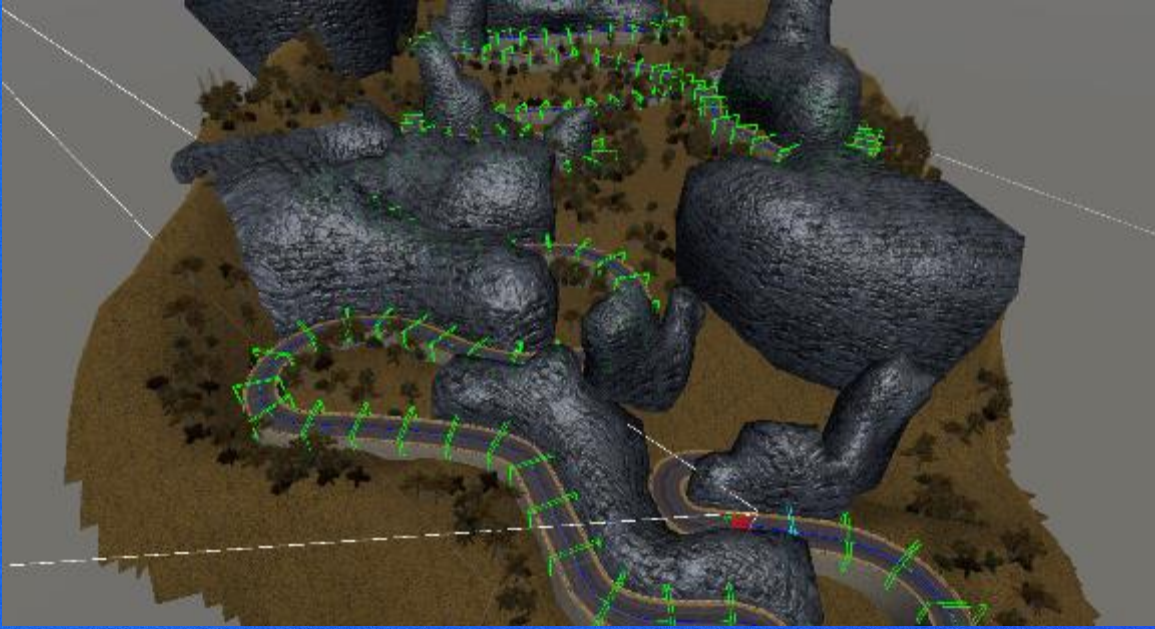
상태 감지
에이전트의 다음 구간까지의 거리,
체크포인트 충돌 처리
통과한 체크포인트가 올바른지 판단

CarAgent

체크포인트 보상 수령
체크포인트와의 거리를 계산해서,
가까워지면 보상(+) 멀어지면 패널티(-)
주행 패널티 전담(역주행, 정체)

게임 개발 내용

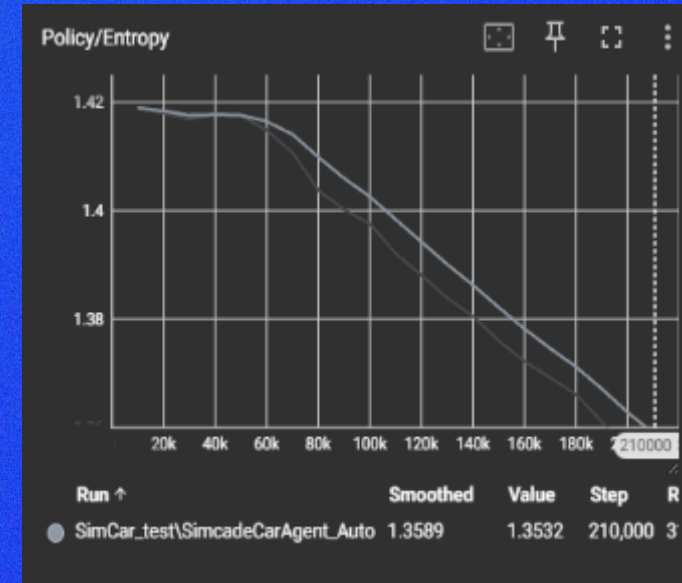
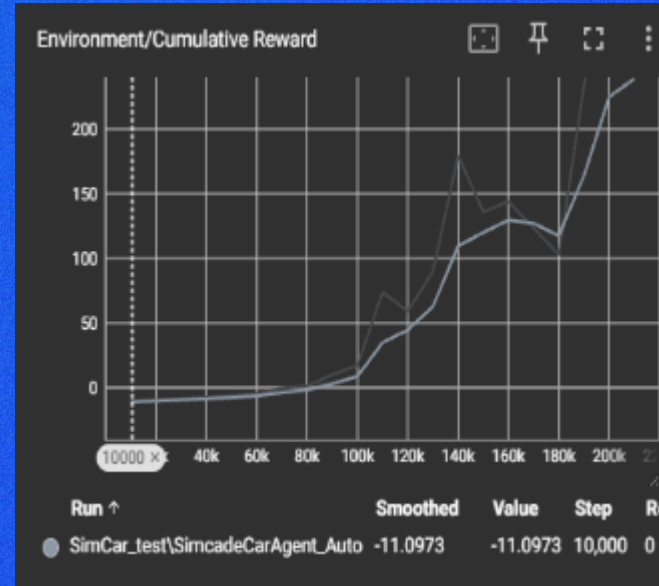
문제 해결 결과



처음에 Spline을 트랙에 따라 연결해 주고, SplineToCheckpointGenerator을 통해 구간(체크포인트)를 자동으로 만들어 줄 수 있었다.
구간(체크 포인트)을 만들어 지나가면 보상값을 받게됨
해당 체크 포인트의 인덱스 값을 CarProgress가 저장/관리
CarAgent는 CarProgress의 체크포인트 인덱스값에 의존

게임 개발 내용

문제 해결 결과

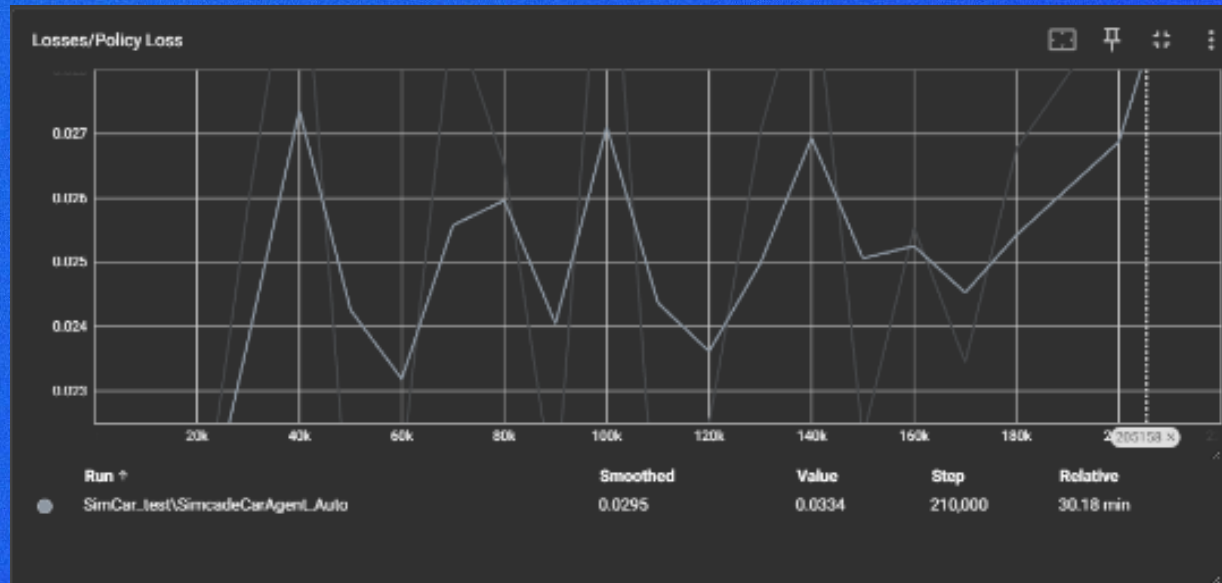


전보다 개선된 주행결과를
보여줌

정상적인 누적 보상의 상향 곡선과
엔트로피의 점진적인 감소가 눈에
띄게 좋아졌음을 확인할 수 있었다.

게임 개발 내용

느낀 점/개선해야 할 점



정상적인 주행을 보여줬지만, 학습이 다소 불안정한 지표결과가 나왔다.

보상 체계가 다소 엄격하여 해당 결과가 나왔다고 생각한다.

게임 개발 내용

느낀 점/개선해야 할 점

보상(+)과 패널티(-)의 극심한 차이, 하이퍼 파라미터의 Batch Size나 학습률의 문제일 수도 있겠다는 생각이 들었다.

급격한 브레이크나, 회전을 막기위해 핸들의 제한을 뒀는데, 이런 클리핑 때문에, 입력제한으로 알고리즘에 예측이 계속 틀려버리는 게 아닌지 생각이 들었다.

여러 번 학습을 시도해 보면서 보상 체계를 엄격하게 잡아야, 효율적이라고 생각했는데, 무조건 그렇지 않다는 것을 깨달았다.

게임 개발 내용

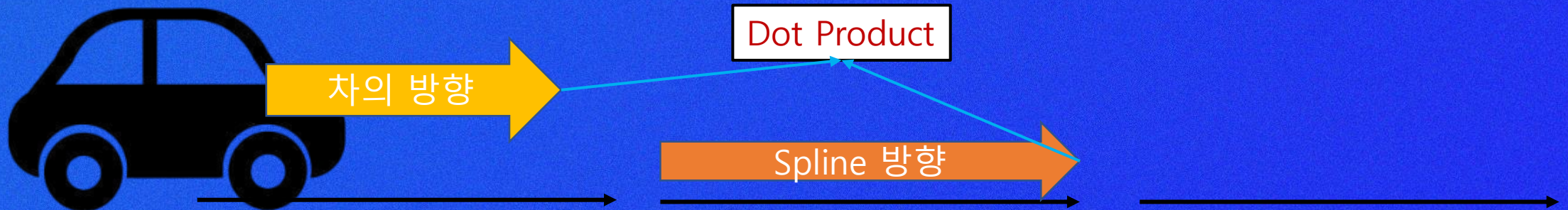
느낀 점/개선해야 할 점

현재 환경은 체크포인트 기반으로 동작하는데, 여기서 체크포인트가 존재하지 않는 환경이면, 주행이 어려울 것이다.

Spline이 가리키는 방향과, 에이전트의 속도 벡터를 단위 벡터로 만들고, 이를 내적한 결과는 $-1.0 \sim 1.0$ 값이 나올 것이다.

이 때 양의 값일 수록 Spline 방향과 일치하고, 음의 값일 수록 Spline 방향과 반대방향이 된다.

이 특징을 활용하면 굳이 체크포인트마다 구간을 지나갔는지 콜라이더 검사를 할 필요 없으니, 좋을 것 같다는 생각이 들었다.



World First Kill

플랫폼	모바일(Android)	장르	RPG, 시뮬레이션
개발 인원	9인(기획1,아트4,플밍4)	개발 기간	2025. 03 ~ 2025. 09
개발 도구	C#, Unity	동영상 링크	https://www.youtube.com/watch?v=EWD2YidSgp0
요약	매 한 판 한 판이 달라지는 RPG의 재미를 줄 수 있는 게임이다. 매 판마다 등장하는 맵, 퀘스트, 직업 정보가 달라지므로, 이런 상황적 랜덤성을 이용해 플레이어는 상황에 맞는 전략과 빌드를 선택해서 게임을 즐길 수 있다.		
역할	서버로 부터 받아온 CSV 데이터 관리/파싱 시스템과 CSV 버전 관리 퀘스트 수락과 보상, 클리어 시스템 맵 이동 시스템 Seed/Token 기반 세이브/로드 시스템		

World First Kill

문제점

방대한 저장데이터를 Json으로 저장해서 불러올 수 밖에 없는 건가?

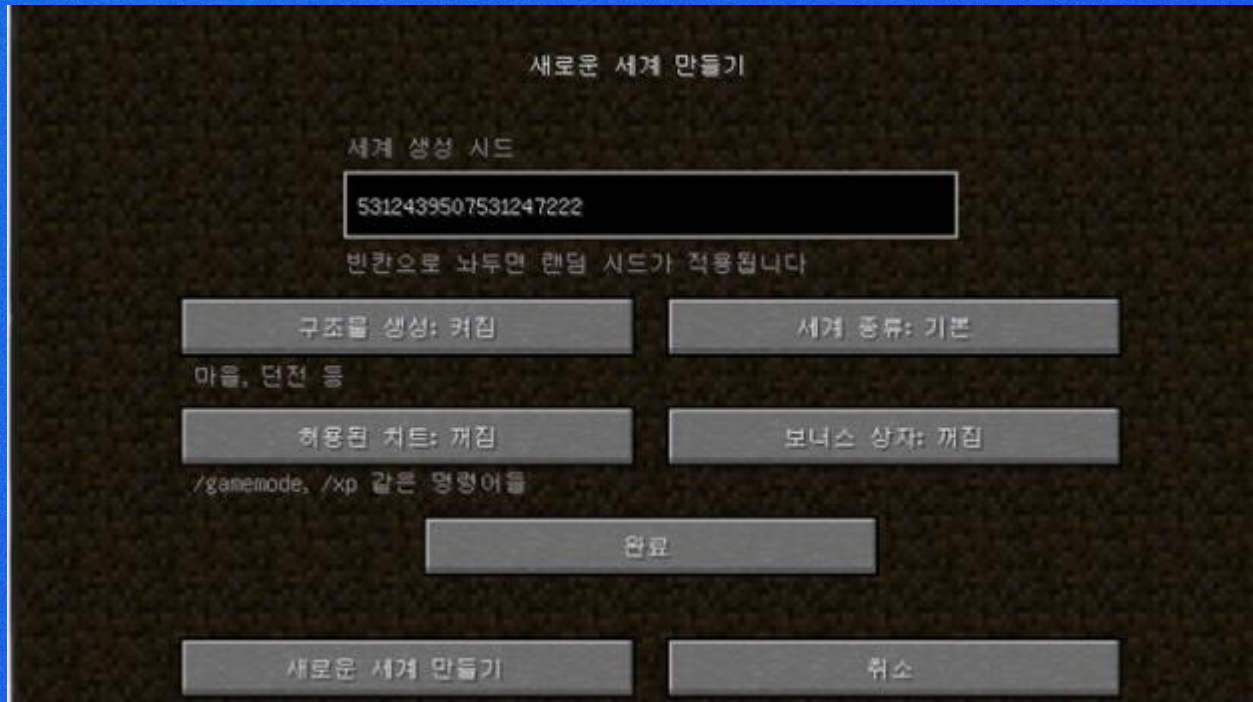
이미 플레이어가 소지하고 있는 아이템은 별도로 저장하고 있는 상태에서 던전, 직업 스킬 및 조합, 퀘스트, 맵 정보를 다 Json으로 저장하면 데이터 크기가 커지고 관리하기 번거롭다는 생각이 들었다.

좀 더 간단하게 "랜덤하게 만들어지는 맵, 퀘스트, 직업 정보 데이터를 들고 올 수 있는 방법이 없을까?" 라는 고민을 하게 되었다.

World First Kill

문제에 대한 해결 기획

마인크래프트의 Seed형 맵 생성을 떠올라, 우리도 랜덤성이 강조된 게임이니, Seed/Token 형식의 맵 생성, 퀘스트 생성, 보상 난수를 불러오자



마인크래프트에서는 해당 시드로, 지형, 환경, 구조물 등을 하나의 코드형태로 생성할 수 있으며 특정 시드를 불러오면, 그 맵의 형태를 그대로 가져올 수 있다.

이를 위해 의사 난수 생성 알고리즘(PRNG)을 참고해서 만들어야 겠다.

World First Kill

문제 해결 과정 설계

난수 대상 조사

지정 도메인	난수 대상	저장되는 곳
Map	지역(region)의 타입, 지역 레벨,	regionType, regionLevel
Region	해당 지역에 생성될 퀘스트 개수, 해당 지역에 나타나는 몬스터 타입	questCount, monsterType
Field	월드 배경 이미지, 필드 내 이벤트 발생 확률	backGroundIndex, randomEventtype
Quest	퀘스트 난이도, 퀘스트 레벨, 퀘스트 종류	difficultyTag, questLevel, questType
Reward	보상의 비율(골드를 더, 경험치를 더)	rewardGold, rewardExp
Dialog	퀘스트 수락/완료 시 표시될 NPC대사	randomDialogIndex
Class	직업 역할군별로 등장할 상위 직업 클래스	PlayerData 내의 class 배열
Skill	각 직업이 초기/후기에 배우게 될 스킬의 조합과 순서	PlayerData 내의 Skill 배열

World First Kill

문제 해결 과정 설계

난수로 어떻게 불러올 수 있을까?

단 하나의 변수인 BaseSeed와 토큰의 조합으로 게임에 난수들을 그대로 불러오는 방법을 사용한다.

Random 함수에 넣어지는 seed를 넣고 돌리는 랜덤성은 결정화된 랜덤성으로, 해당 시드와 Random의 사용량에 영향을 받아 원래 랜덤성을 그대로 가져와야 한다.

즉, 해당 난수의 사용 누적량을 가져와서 무작위성의 흐름을 이어서 유지하기 위해 토큰을 구성한다.

Splitmix64(PRNG)를 참고 사용하여 난수를 생성한다. Random(seed+1), (seed+3) 이렇게 랜덤성을 부여하면 랜덤성의 유사도가 높게 나오는 경우가 있기 때문에 하나의 숫자라도 달라지면, 격하게 랜덤성이 바뀌어 버리는 알고리즘을 채택해서, 다양한 랜덤성을 부여한다.

World First Kill

문제 해결 과정 설계

게임에서 사용하는 난수(Seed) 관리자

GameSeed

- +도메인 enum
- +baseSeed(대표 Seed)
- +난수 사용량을 저장할 딕셔너리
- +SplitMix64 해시 알고리즘을 사용한 랜덤성 부여

세이브/로드를 관리하는 매니저

SaveLoadManager

- +BuildSeedToken: 난수 상태 정보를 텍스트(토큰)으로 압축해서 토큰을 생성한다.
- +RestoreUsageCount: 저장되어 있던 텍스트(토큰)을 해독하여, 난수 사용량을 복구한다.
- +SaveSeedTokenToJson: 현재 시드와, 토큰 정보를 Json 파일로 저장/최신화 한다.
- +TryLoadSeedToken: Json 파일에서 토큰을 읽고

World First Kill

문제 해결 과정 설계



World First Kill

문제 해결 과정 설계



World First Kill

문제 해결 결과

```
{
  "baseSeed": 1826533749,
  "seedToken": "00000000001826533749014001000000000028002000000000018003000",
  "updatedAt": "2026-05-19 21:20:54"
}
```

하나의 SeedToken을 이용해서 완벽히 난수 기능을 불러올 수 있었다.

총 64비트 단위의 BaseSeed를 이용하여 세이프/로드가 가능했다.

그림 자료를 기준으로 BaseSeed는 1826533749이다.

이제 해당 Seed로 세이프/로드를 할 수 있으며 일일이 데이터를 저장해서 Json파일이 커지는 것을 예방할 수 있다.

DATA ENCODING MAP

[BASE SEED]

[COUNT]

[ID]

[USAGE PROGRESS...]

0001234567 (10자)

뱅크 규칙을 결정하는 고유 번호

003 (3자)

저장된 도메인(뱅크, 상점 등)의 개수

001 (3자)

현재 읽고 있는 시스템 식별 번호

0000000000042 (12자)

해당 시스템의 난수 사용 누적량

World First Kill

문제 해결 결과

각기 다른 시드이면, 아래와 같은 이미지 자료처럼, 다른 스킬셋 분포를 가지게 된다. (스킬은 난수로 판정짓기 때문) 이 때문에 개발자가 해당 스킬에 대한 정보까지 저장하지 않아도 그대로 정보를 불러올 수 있음에 간편하다



```
{
  "baseSeed": 1459589277,
  "seedToken": "000000000014595892770140010000000000260",
  "updatedAt": "2026-05-19 21:27:06"
}
```

```
{
  "baseSeed": 1459589276,
  "seedToken": "000000000014595892760140010000000000270020000000",
  "updatedAt": "2026-05-19 21:27:51"
}
```


World First Kill

느낀점/개선할 점

현재 Seed/Token기반의 가장 문제점은 "초기화 순서가 무조건 맞아야 세이브/로드가 성립한다" 이다. 또한 현재 난수의 종류가 늘어나면 늘어날수록, 해당 토큰의 길이가 감당이 안 될 만큼 길어지게 될 수 있으므로, 이점 또한 유지보수 측면에서 개선해야 할 점이다. 해당 문제를 개선하기 위해 아래와 같은 전략을 취할 수 있을 것 같다.

String 기반 난수로 전환하기

기존 방식에서, 난수의 순번이 꼬이면, 로드할 때 이상한 값이 들어올 수 있는 위험성이 존재한다. 또한 토큰의 길이, 그리고 초기화 순서가 맞아야 한다는 단점이 존재했다.

문자열 해시 기반으로 간다면, 퀘스트 고유 번호나 맵의 레벨, 직업의 스킬 인덱스 등을 하나의 문자열 형태로 만들어, 하나의 고유 주소처럼 만드는 것이다.

예를들어, Quest_{Quest_Type}_{QuestID} 이런식으로 조합하여 고유 문자열로 만들고, 이를 우리 BaseSeed와 같이 SplitMix64 알고리즘에 넣어 고유한 시드 숫자로 변환하는 것이다.

World First Kill

느낀점/개선할 점

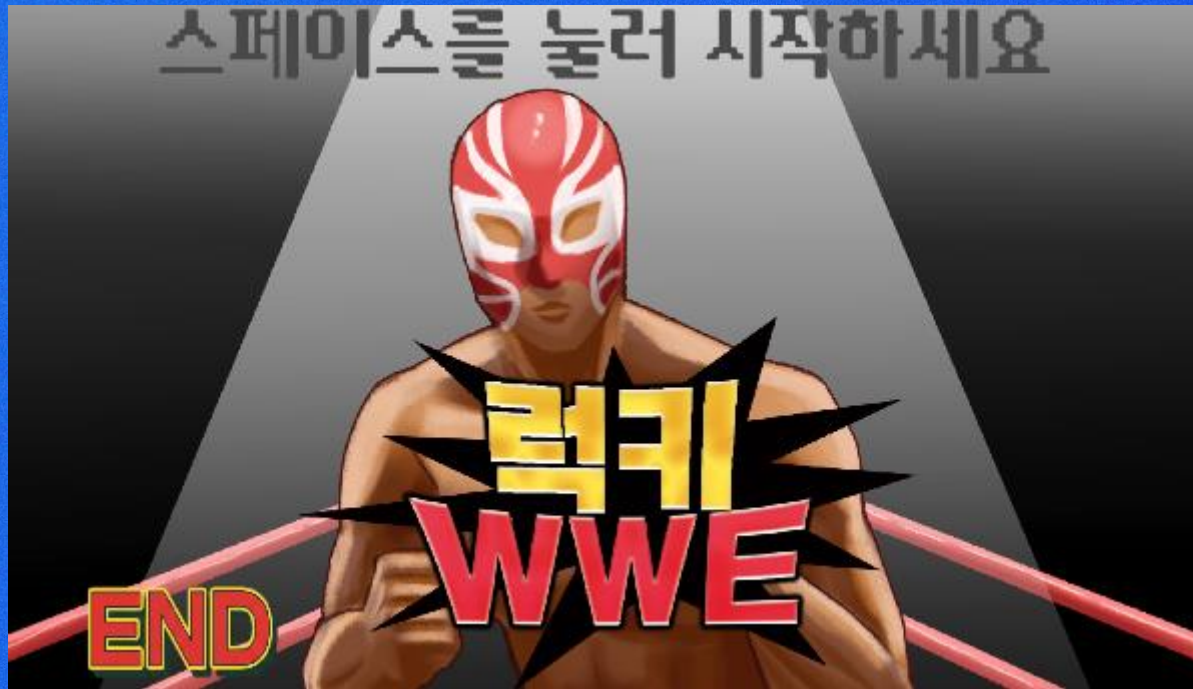
String 기반 난수로 전환 시 이점

- 일일이 난수 사용량을 기록할 필요가 없어진다. 결국 문자열로 고유한 type과 id 등을 조합해서 만드는 것이기 때문에, 사용량을 굳이 기록하여 불러올 필요도, 저장할 필요도 없는 것이다.
- 문자열 + BaseSeed의 조합으로 Token의 길이가 크게 감소하며, 고유한 문자열과 Seed의 조합으로 세이브/로드를 진행하기 때문에, 초기화 순서도 굳이 맞춰줄 필요가 없어질 것이다.

String 해싱 기반 난수로 하면 이와 같은 이점이 있으며, 이젠 Seed하나로 난수 변수를 세팅할 수 있다는 것에 큰 이점이 있는 것 같고 느꼈다.

기타 언급 및 프로젝트

럭키 WWE



플랫폼	PC(Windows)	장르	TCG
개발 인원	4인(기획1,아트2,플밍1)	개발 기간	2026.01.30 ~ 2026.02.01
개발 도구	C#, Unity	동영상 링크	https://www.youtube.com/watch?v=5oXijTkkfQk
요약	Yacht dice 시스템과, WWE 장르의 게임을 섞은 TCG게임 WWE 기술을 Yacht의 콤보와 결합하여 특정 콤보를 넣으면 호응 점수와, 보너스 데미지를 적에게 줄 수 있다.		
역할	카드의 능력 상향이나, 특수 능력을 가질 수 있는 유물 시스템 카드 콤보 조합 시스템 유물, 카드의 데이터 관리 및 적용 로직 유저 편의성 카드 콤보 조합표 제작 전반적인 게임 시스템 전담		

기타 언급 및 프로젝트

Hey! HoneyBee!



플랫폼	PC(Windows)	장르	퍼즐
개발 인원	3인(기획1, 프로그래밍2)	개발 기간	2026.01.10 ~ 2026.01.12
개발 도구	HTML, Verse8	동영상 링크	https://verse8.io/c6C4f9v
요약	꿀 블록을 옮겨가면서 10의 합이 되도록 하는 간단한 퍼즐형 게임이다. 스테이지가 올라갈 수록 퍼즐 크기가 커지고, 잠금 노드, 꿀단지 노드 등이 등장해서 게임의 재미감을 올릴 수 있도록 만들었음		
역할	기획 참여 꿀 블록 이동, 합 연산 계산 로직 꿀단지 노드 제작		

기타 언급 및 프로젝트

집에 돌아가고 싶은 슬라임씨



플랫폼	PC(Windows)	장르	액션 플랫폼머
개발 인원	3인(기획1, 플밍2)	개발 기간	2024.06. ~ 2024.011.
개발 도구	C#, Unity	동영상 링크	https://www.youtube.com/watch?v=83IMfQY1Rwo
요약	액션 플랫폼머 게임으로, 커비를 모티브로 하여 슬라임이 죽인 몬스터의 몸을 빼앗아 능력을 사용할 수 있다. 맵을 탐험하면서 코인을 모아 능력을 향상할 수 있으며, 최종 보스를 깨는 것을 목표로 한다.		
역할	플레이어(슬라임)의 이동, 공격, 변신 전반적인 시스템 개발 몬스터에게 상태 패턴을 사용해 몬스터 AI를 설계 보스 몬스터 설계 스테이지 관리 및 이동 시스템 설계		

기타 언급 및 프로젝트

PipeGame



플랫폼	모바일(Android)	장르	퍼즐
개발 인원	1인(개인)	개발 기간	2024.09.10 ~ 2024.09.17
개발 도구	C#, Unity	동영상 링크	https://www.youtube.com/watch?v=iz2g9ujFx-A
요약	파이프 퍼즐 게임으로, 파이프 시작점을 클릭하고, 끝 파이프 점에 도달하여 연결하는 게임 모든 파이프를 연결하면 그대로 스테이지 클리어		
동기	한 번 간단한 게임을 혼자서 만들어보자는 목적으로 만들어 보게 되었다. 자신이 어디까지 기능 구현을 할 수 있으며, 공부 목적으로 혼자 일주일간 개발을 진행해 봤다.		

기타 언급 및 프로젝트

보름달 방앗간



플랫폼	모바일(Android)	장르	시뮬레이션
개발 인원	7인(기획1,아트4,플밍2)	개발 기간	2024.09. ~ 2024.11.14
개발 도구	C#, Unity	동영상 링크	https://www.youtube.com/shorts/Z-IKElImAMo
요약	NPC, 플레이어가 음식을 만들고 서빙하며, 손님에게 음식을 대접하고 재화를 버는 힐링형 방치 게임		
역할	아이템 카테고리별 인벤토리 시스템 손님,직원 길찾기 시스템 애니메이션 적용, 직원 일 처리(서빙, 음식 제작 등) 시스템		

기타 언급 및 프로젝트

WinAPI Shooting Game



플랫폼	PC(Windows)	장르	슈팅게임
개발 인원	2인(플밍2)	개발 기간	2025.05. ~ 2주간 진행
개발 도구	C++, WinAPI	동영상 링크	https://www.youtube.com/watch?v=XoW4OQZUVJs
요약	플레이어는 적 우주선의 탄알을 피해서 최종 보스를 물리치는 것을 목적으로 한다.		
역할	적 AI 시스템 보스, 스킬(보스 탄환 발사 기술) 시스템 플레이어 이동, 공격 시스템 피격 판정 콜라이더 시스템		

감사합니다.